在线推理调度系统：完整技术文档

Quan’t 队内技术分享

Quan’t Team

2026 年 4 月

目录

1 项目概览 **3**

2 目录结构 **3**

3 Mock 评测平台 **4**

3.1 启动方式 4

3.2 四个 HTTP 端口 5

3.2.1 `/register` —注册队伍 5

3.2.2 `/query` —查询可用任务概览 5

3.2.3 `/ask` —接受任务，获取完整数据 6

3.2.4 `/submit` —提交推理结果 7

3.3 任务生成逻辑（`task_generator.py`） 8

3.4 得分计算逻辑（`scorer.py` + `config.py`） 8

3.5 调试端口（Mock 专用） 9

4 参赛服务：整体架构 **9**

5 参赛服务：模块详解 **11**

5.1 `client.py` —平台 HTTP 客户端 11

5.2 `scheduler.py` —准入控制器 11

5.2.1 LatencyTracker —延迟历史数据库 11

5.2.2 Scheduler.should_accept() —准入决策 12

5.2.3 SGLang 队列信息的刷新 13

5.3 `inference.py` —SGLang 推理客户端 13

5.3.1 generate_until 13

5.3.2 loglikelihood 14

5.3.3 loglikelihood_rolling 14

5.3.4 process_messages —并发处理 14

5.4 `main.py` —主控逻辑 15

5.4.1 常量配置 15

5.4.2 优先级公式 15

5.4.3 Poller 主循环 15

5.4.4 Dispatcher 协程 16

5.4.5 handle_task —任务生命周期 16

5.5 dashboard.py —实时监控 17

6 部署流程（竞赛环境） 18

6.1 setup.sh ——一次性安装（结果被缓存） 18

6.2 run.sh —每次启动执行 18

6.2.1 第一阶段：GPU 架构检测 18

6.2.2 第二阶段：启动 SGLang 19

6.2.3 第三阶段：启动参赛服务 19

7 推理加速技术 19

7.1 张量并行（tp=4） 19

7.2 连续批处理（Continuous Batching） 20

7.3 Chunked Prefill 20

7.4 RadixAttention 20

8 得分策略总结 20

9 常见问题 21

1 项目概览

本项目是为在线大模型推理评测竞赛开发的参赛系统，核心目标是：**在固定时长内，最大化 SLA 达成率和总得分**。

整个 codebase 分为两个独立子系统：

子系统	目录	用途
Mock 评测平台	`mock_platform/`	本地开发调试用，模拟真实竞赛平台行为
参赛选手服务	`contestant/`	正式参赛提交的推理调度服务

---

2 目录结构

```
quant/
├── mock_platform/          # 模拟评测平台（本地调试用）
│   ├── server.py           # FastAPI HTTP 服务，实现 4 个竞赛端口
│   ├── task_generator.py   # 任务生成器（24 道题库 × 随机 SLA）
│   ├── scorer.py           # 得分 / 扣分计算逻辑
│   ├── config.py           # SLA 等级、采样参数、权重常量
│   └── mock_config.json    # 本地运行配置（模型路径、时长等）
│
├── contestant/             # 参赛服务
│   ├── run.sh              # 竞赛启动脚本（GPU检测→SGLang→main）
│   ├── setup.sh            # 一次性依赖安装脚本
│   ├── main.py             # 主循环：poller + dispatcher + handle_task
│   ├── scheduler.py        # 准入控制：EWMA + queue_factor
│   ├── inference.py        # SGLang 推理客户端（3 种任务类型）
│   ├── client.py           # 平台 HTTP 客户端（register/query/ask/submit）
│   ├── dashboard.py        # 实时监控 Dashboard（rich）
│   ├── config_loader.py    # 读取 mock_config.json
│   └── visualizer.py       # 会话结束后保存图表
```

```
│
└── report/                    # 技术报告
    ├── report_zh.md / .pdf    # 对外技术报告（中文）
    ├── report_en.md / .pdf    # 对外技术报告（英文）
    └── team_report_zh.md      # 本文档
```

---

3 Mock 评测平台

Mock 平台的作用是**在本地复现竞赛平台的完整行为**，让我们在没有真实平台的情况下调试和测试调度逻辑。

3.1 启动方式

```
# 终端 1：启动 mock 平台
uvicorn mock_platform.server:app --host 0.0.0.0 --port 8003

# 终端 2：启动 SGLang（本地小模型调试）
python -m sglang.launch_server \
    --model-path /path/to/Qwen3-0.6B \
    --host 0.0.0.0 --port 30000

# 终端 3：启动参赛服务
CONFIG_PATH=mock_platform/mock_config.json \
TEAM_TOKEN=mytoken \
SGLANG_URL=http://localhost:30000 \
python -m contestant.main
```

配置文件 `mock_platform/mock_config.json`：

```
{
  "platform_url": "http://localhost:8003",
  "model_name":   "Qwen3-0.6B",
  "model_path":   "/path/to/Qwen3-0.6B",
```

```
  "duration_s":   3600,
  "sla_levels": {
    "Bronze":   {"ttft_avg": 10.0},
    "Supreme":  {"ttft_avg": 0.5}
  }
}
```

修改 `model_name` / `model_path` 即可切换本地模型；`duration_s` 控制一次 session 的时长。

3.2 四个 HTTP 端口

所有接口均为 `POST`，Mock 服务器与真实竞赛平台接口完全一致：

3.2.1 `/register` —注册队伍

```
// 请求
{"name": "Quan't", "token": "secret_token"}

// 响应
{"status": "ok"}
```

注册后平台初始化该 token 对应的得分条目。

3.2.2 `/query` —查询可用任务概览

```
// 请求
{"token": "secret_token"}

// 响应（任务概览，不含完整数据）
{
  "task_id": 42,
  "target_sla": "Diamond",
  "target_reward": 0.0,
```

```
  "eval_task_name": "mock_generate_until",
  "eval_request_type": "generate_until",
  "eval_sampling_param": "Deterministic",
  "eval_timeout_s": 600
}

// 无任务时
HTTP 404
```

Mock 服务器从可用任务池**随机**返回一个概览（而非 FIFO），防止客户端在同一个任务上无限重试。

3.2.3 `/ask` —接受任务，获取完整数据

```
// 请求
{"token": "secret_token", "task_id": 42, "sla": "Diamond"}

// 响应（接受成功）
{
  "status": "accepted",
  "task": {
    "overview": { ... 同 query 返回... },
    "messages": [
      {
        "ID": 0,
        "prompt": "Question: ...",
        "eval_req_id": "w0_a3f2b1c4",
        "eval_request_type": "generate_until",
        "eval_gen_kwargs": {"until": ["\n\n"], "max_gen_toks": 32, "temperature": 0.0, ...},
        "eval_continuation": null
      }
    ]
```

```
  }
}

// SLA 不匹配时
{"status": "rejected", "reason": "SLA must match target"}

// 任务已被其他人抢走
{"status": "closed"}
```

重要：ask 时传入的 sla **必须与 query 返回的 target_sla 完全一致**，否则被拒绝。

3.2.4 /submit —提交推理结果

```
// 请求
{
  "user": {"name": "Quan't", "token": "secret_token"},
  "msg":  {
    "overview": { ... 原始 overview dict... },
    "messages": [
      {
        "ID": 0,
        "prompt": "...",
        "response": "Au",          // generate_until: 填 response
        "accuracy": null,           // loglikelihood: 填 accuracy (log prob)
        "eval_request_type": "generate_until",
        ...
      }
    ]
  }
}
```

```
// 响应
{"status": "ok"}
```

3.3 任务生成逻辑（task_generator.py）

题库共 24 道题（8 SLA × 3 任务类型），每道题对应固定的 SLA 等级。每次 query 请求触发一次 random.choice(PROMPT_POOL)，再随机抽取采样参数：

```
SAMPLING_PARAM_DISTRIBUTION = ["Deterministic", "Deterministic", "Normal", "HighEntropy"]
# 50% Deterministic, 25% Normal, 25% HighEntropy (ExtremePenalty 不在本次 mock 中出现)
```

三种任务类型的 message 结构不同：

任务类型	messages 数量	关键字段
generate_until	1 条	eval_gen_kwargs（含 until、max_gen_toks、temperature 等）
loglikelihood	4 条（4 个选项）	eval_continuation（选项文本），eval_gen_kwargs=null
loglikelihood_rolling	1 条	无 eval_gen_kwargs，无 eval_continuation

---

3.4 得分计算逻辑（scorer.py + config.py）

得分公式（在 SLA 时限内提交）：

$$R_i = w_{\text{task}} \times w_{\text{sla}} \times w_{\text{sp}} \times C_i$$

变量	含义	取值
w_{task}	任务类型权重	generate_until=2.0，loglikelihood=1.0，rolling=1.0
w_{sla}	SLA 等级权重	Bronze=1.0 → Supreme=2.5
w_{sp}	采样参数权重（仅 generate_until）	Deterministic=1.0，Normal=1.1，HighEntropy=1.2
C_i	正确性得分	Mock 阶段固定为 1.0；正式比赛与参考模型对比

三种提交时机对应的结果：

```
提交时间 ≤ SLA TTFT    →  正得分  R_i
SLA TTFT < 提交时间 ≤ 600s  →  0 分（不得也不扣）
提交时间 > 600s        →  扣分  -2 × R_i
```

3.5 调试端口（Mock 专用）

```
# 查看当前得分
GET http://localhost:8003/scores


# 查看任务队列状态
GET http://localhost:8003/status
```

---

4 参赛服务：整体架构

```
┌──────────────────────────────┐
│      评测平台 (HTTP)          │
│  /query /ask /submit         │
└──────────┬──────────────────┘
```

```
                    │
┌──────────────────────▼──────────────────────┐
│              contestant.main                 │
│                                              │
│  □  Poller Loop                              │
│     while time < deadline:                   │
│       overview = query()                     │
│       if should_accept(overview):            │
│         task = ask(task_id, sla)             │
│         enqueue(priority_queue, task)        │
│                                              │
│  □  Dispatcher Coroutine                     │
│     while True:                              │
│       task = await priority_queue.get()      │
│       create_task(handle_task(task))         │
│                                              │
│  □  handle_task(task) × N 并发               │
│     results = await gather(*messages)        │
│     await submit(results)                    │
│     scheduler.latency.record(elapsed)        │
└──────────────────────┬──────────────────────┘
                       │ HTTP
┌──────────────────────▼──────────────────────┐
│         SGLang 推理后端 :30000               │
│  tp=4 · 连续批处理 · chunked prefill         │
│  RadixAttention · priority scheduling        │
└──────────────────────────────────────────────┘
```

5 参赛服务：模块详解

5.1 client.py —平台 HTTP 客户端

封装四个端口，使用 `httpx.AsyncClient`（异步非阻塞）：

```
class PlatformClient:
    async def register() -> bool
    async def query()   -> Optional[dict]   # None = 无任务（404）
    async def ask(task_id, sla) -> Optional[dict]  # None = rejected/closed
    async def submit(overview, messages) -> bool
```

`ask()` 里有一个重要细节：平台对 SLA 不匹配会返回 `{"status": "rejected"}`，而不是 HTTP 错误码，需要在应用层判断。

5.2 scheduler.py —准入控制器

这是整个系统**最核心的决策模块**，负责回答："这个任务现在接不接?"

5.2.1 LatencyTracker —延迟历史数据库

```
class LatencyTracker:
    ALPHA = 0.3
    _ewma:        dict[(task_type, sla), float]   # 调度决策用
    _sla_samples: dict[(task_type, sla), deque]   # Dashboard 展示用（窗口 50）
```

每个 `(task_type, sla)` 组合独立追踪，防止不同类型的延迟互相污染。例如 `("generate_until", "Supreme")` 和 `("loglikelihood", "Bronze")` 完全独立。

每次任务完成后调用 `record(task_type, sla, elapsed)` 更新：

$$\mathrm{EWMA}_t = 0.3 \times \mathrm{elapsed}_t + 0.7 \times \mathrm{EWMA}_{t-1}$$

除了 EWMA，还维护滑动窗口用于 Dashboard 计算 P95 / 平均延迟 / SLA 命中率。

5.2.2 Scheduler.should_accept() —准入决策

```
def should_accept(self, overview: dict) -> bool:
    sla       = overview["target_sla"]
    task_type = overview["eval_request_type"]

    # □ 硬上限：超过 max_concurrent=64 个并发任务直接拒绝（兜底保护）
    if self.active_count >= self.max_concurrent:
        return False

    # □ 冷启动：EWMA 还没数据，跳过延迟检查，直接接受（探针模式）
    ewma = self.latency.ewma_latency(task_type, sla)
    if ewma is None:
        return True

    # □ 核心公式：队列深度感知的延迟估算
    W = self._sglang_waiting   # SGLang 排队等待的任务数
    R = self._sglang_running   # SGLang 正在 GPU 上运行的任务数
    queue_factor = W / R       # W/R：新任务预计等待几个推理周期
    estimated = ewma * (1 + queue_factor)
    sla_limit = SLA_TTFT[sla]

    if estimated >= sla_limit:
        # □ 探针机制：连续拒绝 8 次后强制接受一次，防止 EWMA 永久过时
        count = self._consec_rejects.get((task_type, sla), 0) + 1
        self._consec_rejects[(task_type, sla)] = count
        if count >= PROBE_THRESHOLD:  # = 8
            self._consec_rejects[(task_type, sla)] = 0
            return True   # 强制接受（探针）
        return False

    self._consec_rejects[(task_type, sla)] = 0
```

```
    return True
```

为什么用 W/R 而不是 active_count/max_concurrent？

旧方案（load_ratio）：- `active_count` 是我们逻辑上"在飞"的任务数，是个代理指标 - 固定的 `max_concurrent=24` 对任务长度完全盲目：24 个 loglikelihood 和 24 个 generate_until 对 GPU 的压力天差地别

新方案（queue_factor）：- `W` 和 `R` 是 SGLang 内部的实时状态，直接反映 GPU 饱和度 - 任务长度感知：generate_until 耗时长 → 占满 R、推高 W → W/R 自然升高 → 准入自动收紧 - 基于排队理论（M/M/c 近似）：W/R 近似为新任务等待的推理周期数

5.2.3 SGLang 队列信息的刷新

`_sglang_waiting` 和 `_sglang_running` 每 0.5 s 从 SGLang 的 `/server_info` 接口拉取（由 dashboard 协程负责，之所以不在 poller 里拉是为了避免增加 query→ask 的关键路径延迟）：

```
# dashboard.py 每个 tick (0.5s) 更新一次
info = await inference.server_info()   # GET /server_info
scheduler.update_sglang_queue(
    info.get("num_waiting_reqs", 0),
    info.get("num_running_reqs", 1),
)
```

5.3 `inference.py` —SGLang 推理客户端

封装对 SGLang HTTP API 的三种调用：

5.3.1 generate_until

```
payload = {
    "model":       model_name,
    "prompt":      prompt,
    "max_tokens":  gen_kwargs["max_gen_toks"],   # 默认 32 (mock), 竞赛可能更长
```

```
    "temperature": gen_kwargs["temperature"],
    "top_p":       gen_kwargs["top_p"],
    "top_k":       gen_kwargs["top_k"],
    "stop":        gen_kwargs["until"],
    "priority":    priority,   # 0-7, 对应 SLA 等级
}
POST /v1/completions → choices[0]["text"]
```

5.3.2 loglikelihood

```
# 把 prompt + continuation 拼接, 请求 input_token_logprobs
full_text = prompt + continuation
payload = {
    "text": full_text,
    "sampling_params": {"max_new_tokens": 1, "temperature": 0.0},
    "return_logprob":    True,
    "logprob_start_len": 0,
    ...
}
POST /generate → sum(input_token_logprobs[-continuation_token_count:])
```

continuation 的 token 数量优先用 tokenizer 精确计算，无 tokenizer 时退回 `len(continuation) // 4`。

5.3.3 loglikelihood_rolling

```
# 整个 prompt 的总 logprob (跳过第一个 token)
payload = {"text": prompt, "return_logprob": True, ...}
POST /generate → sum(input_token_logprobs[1:])
```

5.3.4 process_messages —并发处理

```
async def process_messages(self, messages, priority):
    return list(await asyncio.gather(
```

```
        *(_process_one(msg) for msg in messages)
    ))
```

同一任务的所有 messages **同时**发往 SGLang。对于 loglikelihood 任务（4 条 messages = 4 个选项），这触发 SGLang 的 RadixAttention：4 条请求共享相同的 prompt 前缀，KV Cache 只计算一次，4 个 continuation 并行 forward。

5.4 main.py —主控逻辑

5.4.1 常量配置

```
# SLA → 计分权重（与 scorer.py 保持同步）
SLA_WEIGHTS = {"Bronze": 1.0, ..., "Supreme": 2.5}
TASK_WEIGHTS = {"generate_until": 2.0, "loglikelihood": 1.0, ...}

# SLA → SGLang 内部 priority（0-7）
SLA_SGLANG_PRIORITY = {"Bronze": 0, ..., "Supreme": 7}
```

5.4.2 优先级公式

```
def _task_priority(overview) -> float:
    return -(SLA_WEIGHTS[sla] * TASK_WEIGHTS[task_type])
```

取负数使 Python 小根堆变为大根堆，得分越高的任务越先出队。最高优先级：Supreme + generate_until = -5.0；最低：Bronze + loglikelihood = -1.0。

5.4.3 Poller 主循环

```
while loop.time() < deadline:
    overview = await platform.query()      # □ 查询任务概览
    if overview is None:
        await asyncio.sleep(0.1); continue

    if not scheduler.should_accept(overview):  # □ 准入决策
```

```
        dash_state.rejected += 1
        await asyncio.sleep(0)            # yield 事件循环，立即重试
        continue

    task_data = await platform.ask(task_id, sla)  # □ 接受任务
    if task_data is None: continue        # 被抢走或 SLA 不匹配

    scheduler.mark_active(task_id)
    await task_queue.put((priority, seq, task_data, ...))  # □ 入队
```

注意 `asyncio.sleep(0)` 的用法：拒绝后不等待，立即重试 query，但让事件循环有机会执行其他协程（防止 busy loop 阻塞 I/O）。

5.4.4 Dispatcher 协程

```
async def dispatcher(task_queue, ...):
    while not stop_event.is_set() or not task_queue.empty():
        item = await asyncio.wait_for(task_queue.get(), timeout=0.1)
        _, _seq, task_data, overview, deadline_time, sglang_priority, dash_state = item
        asyncio.create_task(handle_task(...))   # 非阻塞，立即返回
```

dispatcher 只负责从队列取出并**创建任务**，不等待任务完成。所有 handle_task 协程并发运行于同一个 asyncio 事件循环。

5.4.5 handle_task —任务生命周期

```
async def handle_task(...):
    t_start = loop.time()
    try:
        result_messages = await inference.process_messages(messages, priority)
        elapsed = loop.time() - t_start

        # 无论是否超期，都提交（避免 -2× 扣分）
        if loop.time() > deadline_time:
```

```
            log.warning(f"SLA MISSED ({elapsed:.2f}s), submitting anyway")

        ok = await platform.submit(overview, result_messages)

        # 更新 EWMA（无论 SLA hit/miss，都用实际延迟更新）
        scheduler.latency.record(task_type, sla, elapsed)

    finally:
        scheduler.mark_complete(task_id)   # 释放并发槽
```

超期仍提交的逻辑是有意设计的：

行为	结果
SLA miss 但在 600s 内提交	0 分，不扣分
600s 内不提交（超时）	$-2 \times$ reward 扣罚
选择"超期就放弃"	必然触发 600s 扣罚

所以放弃提交是严格劣势策略。

5.5 dashboard.py —实时监控

基于 `rich.live.Live` 渲染，每 0.5 s 刷新：

```
┌──────── Quan't Live Dashboard ──────────────────────────────┐
│ Session: 00:12:34   Accepted: 1024   Rejected: 87   Score: 4821 │
│ Completed: 987   SLA Miss: 12 (1.2%)                          │
├──────────────────────────────────────────────────────────────┤
│ Task Type     │ SLA      │ EWMA    │ P95     │ Hit Rate │ Count     │
│ generate_until│ Supreme  │ 0.31s   │ 0.44s   │ 98.2%    │ 54        │
│ loglikelihood │ Bronze   │ 0.18s   │ 0.29s   │ 100%     │ 312       │
│ ...                                                           │
├──────────────────────────────────────────────────────────────┤
```

```
│ SGLang Queue: waiting=2  running=18   active_tasks=21          │
└──────────────────────────────────────────────────────────────────┘
```

---

6 部署流程（竞赛环境）

竞赛平台按顺序执行两个脚本：

6.1 setup.sh ——一次性安装（结果被缓存）

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r contestant/requirements.txt

# 放宽 cuda-python 版本约束（12.9 → ≥12.8），兼容竞赛服务器 CUDA 12.8
sed -i 's/cuda-python==12\.9/cuda-python>=12.8/' sglang/python/pyproject.toml
pip install -e "sglang/python/"

pip install flash-attn --no-build-isolation || true   # 可选，失败不影响运行
```

6.2 run.sh ——每次启动执行

6.2.1 第一阶段：GPU 架构检测

```
ATTN_BACKEND=$(python3 - <<'EOF'
import torch
try:
    major, _ = torch.cuda.get_device_capability(0)
    print("triton" if major >= 12 else "flashinfer")
except Exception:
    print("triton")   # Blackwell (SM 12.x) + CUDA 12.8 会走这里
EOF
)
SMPL_BACKEND=$( [ "${ATTN_BACKEND}" = "triton" ] && echo "pytorch" || echo "flashinfer" )
```

RTX 5090（Blackwell，SM 12.0）在 CUDA 12.8 下有两处 FlashInfer JIT 不兼容：

报错位置	原因	解决方案
`flashinfer/jit/core.py::check_cuda_arch()`	`get_device_capability()` 返回 None，被判定为 SM < 75	`--attention-backend triton`
`flashinfer/sampling.py::get_sampling_module()`	同上，独立的 JIT 路径	`--sampling-backend pytorch`

6.2.2 第二阶段：启动 SGLang

```
python -m sglang.launch_server \
    --model-path "${MODEL_PATH}" \
    --host 0.0.0.0 --port 30000 \
    --tp-size "${TP_SIZE}" \           # 4 卡时为 4
    --schedule-policy fcfs \
    --enable-priority-scheduling \     # 开启内部优先级调度
    --chunked-prefill-size 4096 \      # 长 prompt 分块，避免阻塞高优先级任务
    --attention-backend "${ATTN_BACKEND}" \
    --sampling-backend "${SMPL_BACKEND}" &

# 轮询 /health，最多等待 55 秒
```

6.2.3 第三阶段：启动参赛服务

```
SGLANG_URL=http://localhost:30000 python -m contestant.main
```

---

7 推理加速技术

7.1 张量并行（tp=4）

Qwen3-32B 参数量约 64 GB（BF16），单卡无法容纳。4 卡张量并行：- 每卡持有 1/4 权重（~16 GB）- 每次 forward pass 通过 NCCL All-Reduce 同步激活值 - 剩余约 16 GB/卡用于 KV Cache，支持大量并发序列

7.2 连续批处理（Continuous Batching）

SGLang 实现 Iteration-level 连续批处理：- 每个 decode step，所有正在生成的序列合并为一个 batch - 某个序列生成完毕后立即释放 KV Cache，新请求无需等待整个 batch - GPU 利用率在整个 session 中持续保持高位

7.3 Chunked Prefill

4096 token/chunk，效果：- 长 prompt 不再独占 GPU，每 chunk 之间 decode 请求可插入 - 与 `--enable-priority-scheduling` 协同：高优先级任务在 chunk 间隙插队 - Supreme（0.5s SLA）不会被 Bronze 的长 prefill 阻塞

7.4 RadixAttention

SGLang 将 KV Cache 组织为前缀树，相同 prompt 前缀的请求共享 KV Cache。对 loglikelihood 任务（4 选项 = 4 条 messages，相同 prompt + 不同 continuation）：- `asyncio.gather` 同时发出 4 条请求 - RadixAttention 自动识别共同前缀，只计算一次 prefill - 4 个 continuation 并行 forward，TTFT　单请求延迟

---

8 得分策略总结

理解得分公式后，我们的策略选择可以更有针对性：

$$\text{总分} = \sum_{\text{SLA hit}} w_{\text{task}} \times w_{\text{sla}} \times w_{\text{sp}} \times C_i$$

高价值任务优先：Supreme generate_until 单任务得分是 Bronze loglikelihood 的 5×，因此调度队列优先处理高 SLA + generate_until 组合。

拒绝低价值任务以保护高价值任务：当 SGLang 队列积压时（W/R 高），拒绝 Bronze 任务，为即将到来的 Supreme 任务预留 GPU 时间，净收益为正。

超期仍提交（硬性原则）：SLA miss 得 0 分，但放弃提交会扣 −2× 分。因此无论延迟多少，必须提交。

采样参数 HighEntropy 得分最高（generate_until 任务中权重 1.2 vs Deterministic 1.0），但目前我们按平台指定的参数执行，无法主动选择。

---

9 常见问题

Q：`query` 一直返回 404 怎么办？

Mock 平台任务生成速率约 30 tasks/s，正常情况下几乎不会 404。若持续 404，检查 mock 平台进程是否存活。

Q：任务被 `ask` 之后返回 `closed`？

比赛中可能有多个队伍竞争同一任务，`closed` 表示被抢走了。我们的 poller 立即重新 query 即可。Mock 平台只有一个队伍，理论上不会出现 `closed`。

Q：如何验证调度在正常工作？

观察 Dashboard 的 `sglang_waiting` 值：- 正常运行时应维持在 0~5 之间 - 持续 >10 说明我们接任务太激进，queue_factor 没有生效（检查 `/server_info` 是否返回正确的 `num_running_reqs`）

Q：SGLang 日志里出现 OOM 怎么办？

降低 `--max-running-requests`（目前未设置，由 SGLang 自动计算）或者减少 `max_concurrent`。

Q：Blackwell GPU 下 FlashInfer 报错？

`run.sh` 已自动检测并切换到 Triton + PyTorch 后端。如果手动启动 SGLang 需自行加 `--attention-backend triton --sampling-backend pytorch`。

文档版本对应代码 commit `f042924`，Quan't Team，2026 年 4 月。